

Data platform for AI workloads on Azure

Choosing a data platform involves understanding the unique data challenges that these solutions bring. GenAI solutions, especially those built with foundation models, depend on diverse, high-quality data, fast access to scalable data stores that supports vector search. The goal is to meet these needs without adding unnecessary complexity to your architecture. Understanding the principles of effective data pipeline design is essential before evaluating platform options.

When evaluating platform choices, start by asking whether you truly need additional components. Simpler architectures are often faster to deploy, easier to manage, and more cost-effective. Ask yourself:

- Can the model achieve its expected performance using data from a single source?
- Does the source data store already provide the analytics or search capabilities you need?
- Is the source data already structured and indexed for AI or vector search?

If the answer is yes to most of these questions, a complex architecture might not be necessary. For example, databases such as Azure Cosmos DB and Azure SQL Database already support vector data types and vector search natively but they need to be enabled and configured. These capabilities can reduce the need for separate indexing or specialized vector databases, minimizing data movement while improving performance.

As your workload grows and data comes from multiple sources, the platform decision becomes more complex. You may need to consider solutions that support ETL or ELT pipelines, specialized search indexes, and scalable storage for large datasets. Each added capability should serve a clear purpose rather than simply expanding the technology stack.

This article provides guidance on choosing a data platform for workloads where data needs to be stored, processed, or analyzed. The focus is on solutions that support generative AI (GenAI). We highly recommend that you understand the principles of good data pipeline design before you explore the technological capabilities that this article describes. For more information, see [Grounding data design](#).

For recommendations specific to discriminative model training and fine-tuning, see [Training data platform considerations](#).

Considerations for data storage platform

In AI workloads, data usually moves through several stages of storage and processing, guided by pipelines that connect each step. One important stage is the data store that holds information collected and combined from multiple sources. This store allows you to process and refine the data until it's ready for the next stage.

Note

You might not need this component in your architecture. In some cases, you could access data directly from the source systems. However, doing so can lead to performance issues and may overload those systems with AI queries. It can also cause access or reliability challenges. To prevent these issues, it's usually better to copy the data into a dedicated store for aggregation and processing.

When you choose a platform for this store, make sure it follows the same security standards as your source systems, is cost-effective, and works well with ETL, ELT, or EL processing tasks. Your options can range from simple storage solutions to large-scale data platforms, depending on your data volume and performance needs. Look for a storage option that's reliable, scalable, and provides good value for your workload.

Here are a few questions to help guide your choice of data store technology.

Can the platform handle different data formats?

Your data store should be able to store a variety of data formats and, when needed, convert data between them.

For example, if your ingestion pipeline brings in data from both a relational database and a JSON file, it must support structured and semi-structured data. You might want to convert your data into Delta format to enable the richer functionality Delta Lake technology provides. The platform should provide built-in tools for this kind of transformation so that you do not need to write custom code.

Do you expect to store multiple versions of the data?

Data changes over time in both values and structure, and source systems usually store only the current state. If you need historical context, choose a data platform that supports versioning. Without it, you might have to duplicate datasets, which adds complexity.

Versioning has other benefits. In some cases, you may need separate copies of data for different use cases. Each copy can evolve independently, and the platform should manage versioning

across all copies to preserve context for your AI models.

Does the platform have built-in data lifecycle management capabilities?

Data lifecycle management (DLM) helps control growth from creation to deletion. Your platform should automatically remove intermediate copies, manage archived data, and support regulatory retention when needed. Without that, data can grow uncontrollably and that unnecessary volume can make processing difficult. For instance, you might need to rerun preprocessing steps several times to improve data quality. The platform should automatically remove intermediate copies when they are no longer needed.

In other cases, you may need to retain data for compliance or audits. Look for storage options that support cold or archived tiers for rarely accessed data at lower cost.

Does the platform support data governance features?

Auditability is an important aspect for AI workloads. Your platform should maintain audit trails to track data access, ensure privacy, and document data origins. It should also support a data dictionary or catalog that manages metadata, data types, purpose, and lineage, especially when data comes from multiple sources.

How much data do you expect to store?

AI workloads generate large volumes of data, which can grow further with multiple versions and extra metadata. Your data platform should scale efficiently for both storage and throughput, handling high ingestion rates, concurrent writes, and intensive processing without performance degradation.

When choosing a platform, consider the entire workflow, as ingestion and processing often happen at the same time. The system should support parallel processing and frequent data movement, and provide telemetry to give clear insight into read and write performance.

Is this data store critical to your workload's reliability?

Choose a platform that supports reliability and scalability through replication or multiple instances. Many big data stores use controllers that distribute processing automatically and provide failover when an instance becomes unavailable.

Data also needs to be both durable and accessible. Make sure the platform guarantees data integrity, provides accessible APIs, and supports backup or restore capabilities if rebuilding data from scratch would be costly.

Do you have any cost constraints?

After reliability and performance requirements are met, consider how to optimize costs. For many AI workloads, a write-once, read-many pattern is sufficient and helps control expenses. Grounding data should be cost-effective to store and retrieve, even if it doesn't require the same level of responsiveness as a production database. The goal is to balance cost, efficiency, and performance.

Do you need to support data sovereignty or regional compliance requirements?

For workloads that handle regulated or sensitive data, consider deploying in a sovereign cloud, such as Azure Government, Microsoft Azure operated by 21Vianet, or other National Partner Clouds. These environments are designed to meet strict data residency, privacy, and compliance requirements by ensuring that data storage, processing, and access remain within specific jurisdictions.

Sovereign clouds provide greater control and independence over your data, which is often a requirement for sectors such as government, defense, or banking. However, keep in mind that some advanced AI and data platform features might not yet be available in these regions. Review service availability before designing your architecture.

Use Microsoft Purview to maintain data cataloging, classification, and lineage tracking across these environments. For highly confidential workloads, consider using confidential computing and customer-managed keys to strengthen data protection. You must verify that your deployment aligns with regional regulations.

Technology options

 Expand table

Function	Recommended Technologies	Alternatives / Complementary Tools
Multi-format data storage	Azure Data Lake Storage Gen2, Microsoft Fabric Lakehouse, Azure Databricks Lakehouse	Azure Blob Storage, Azure Synapse Analytics, on-premises data warehouse
Data versioning and lineage	Microsoft Fabric Lakehouse, Azure Data Lake Storage Gen2 (with Delta Lake), Azure Databricks (Delta Lake)	Git LFS, DVC (Data Version Control), Apache Iceberg

Function	Recommended Technologies	Alternatives / Complementary Tools
Data lifecycle management (DLM)	Azure Data Lake Storage Gen2 (lifecycle policies), Azure Blob Storage (tiering), Azure Databricks (table optimization)	Amazon S3 (lifecycle policies), Google Cloud Storage
Data governance and cataloging	Microsoft Purview, Azure Databricks Unity Catalog	Apache Atlas, DataHub, Collibra
High-volume data storage	Azure Data Lake Storage Gen2, Azure Synapse Analytics, Azure Databricks Lakehouse	Azure Blob Storage, Hadoop HDFS, Amazon S3

Considerations for data processing platform

The data processing platform plays a key role in preparing and transforming data so it's ready for downstream use, whether that's RAG indexing, analytics, or other use case.

📌 Note

For GenAI and retrieval-augmented generation (RAG), it's useful to understand the difference between ETL, ELT, and EL processes.

- ETL: Extract, transform, then load, typical for traditional data warehousing.
- ELT: Extract, load, then transform, common for data lakes and big data tools like PySpark.
- EL: Extract and load, used in RAG scenarios where you store documents first, then perform transformations like text chunking or image extraction later.

There are two places where processing can happen:

- Ingestion Layer. The ingestion pipeline collects data from various sources and moves it into your aggregate data store. Along the way, it often performs basic preprocessing or formatting so the data is queryable. To reduce the need for custom code, it's best to use a data platform that handles as much of this as possible. When evaluating tools, consider the ETL or ELT features required to support your AI workloads, such as data augmentation.
- Processing Layer. After data lands in the aggregate store, it typically needs deeper processing before it's ready for indexing or use in AI models. These pipelines should offer similar levels of reliability and scalability as your ingestion layer, but the focus shifts to transforming and reshaping the data.

Typical tasks include:

- Entity recognition and enrichment
- Integrating additional data sources
- Performing lookups and transformations
- Cleaning or deleting irrelevant data

A strong data platform helps automate and orchestrate these operations efficiently.

What's the support for connecting to data sources?

The platform should easily connect to data sources you expect ingest from, whether that's relational databases, big data sources, or blob storage.

Look for prebuilt connectors and low-code integrations. Ideally, you want drag-and-drop or configuration-based connectors that support lookups, data copying, and governance.

Can the platform process various data formats?

Data comes in many shapes: structured (SQL, relational tables), semi-structured (JSON, XML, Parquet), and unstructured (documents, images), and streaming (IoT data). Choose a platform that can handle the formats your use case requires considering immediate and longer term requirements.

Does the platform offer features for data preparation and rescoping?

Before your data is ready for indexing or model consumption, it needs to be cleaned, enriched, and reshaped. Your data design strategies should explicitly outline the requirements. A good platform should:

- Remove duplicates and fill missing values
- Handle stemming, normalization and other basic cleanup tasks when planning to support keyword or hybrid (keyword+vector) search
- Support advanced transformations like chunking, enrichment, and document analysis

If your data store supports these operations natively, you can process data in place without moving it. Otherwise, use external tools like Azure Databricks or Azure Data Factory for heavy transformations.

In some cases, you may choose to externalize part of this responsibility to the platform that supports the next stage. A common example of this approach is RAG implementation. During

processing, documents are divided into smaller chunks, with each chunk stored as a separate row in the index. These chunks are then paired with embeddings, often generated through an OpenAI service. In Azure AI Search, this process is orchestrated as part of the enrichment pipeline during indexing, where documents are processed by an embedding model (such as an OpenAI embedding model) to generate vector representations that are then stored in the index.

Is there a built-in orchestrator for managing workflows?

Data processing typically happens as modular jobs that need complex coordination. Your platform should include an orchestrator to define, schedule, and monitor these workflows. Look for:

- Support for job dependencies and checks that validate sequence of execution
- Flexible modification of workflows that allows for easy adjustments without rewriting large portions of code.
- Monitoring and logging capabilities

Popular tools include Azure Data Factory for its rich feature set for workflow management, or Azure Databricks for more complex orchestration. If cost is a concern, Apache NiFi or Airflow can be more economical alternatives.

How much data do you expect to ingest?

Estimate how much data you'll ingest and the frequency of ingestion. For example, if you expect to load 10 terabytes of data daily into an index, the platform should support strong parallelization and distributed execution. For smaller workloads, simpler tools like Logic Apps might work, but for higher volumes, Data Factory or Databricks are more suitable. For scalability and throughput, consider:

- Data volume and frequency
- Tolerable latency requirements
- Job complexity

For instance, data cleansing involves validating and potentially replacing invalid fields or masking sensitive information. These tasks, though basic, require significant resources because each row is processed individually, which adds to the overall time.

What monitoring capabilities do you need?

Data processing pipelines should have monitoring capabilities and provide insights into the pipeline's performance and status of jobs. Your platform should provide:

- Job progress tracking
- Logs, metrics, and alerts to understand pipeline behavior
- Integration with your broader monitoring stack

Identify any gaps in the built-in telemetry, and determine what additional monitoring you need to implement. This monitoring might involve adding custom logging or metrics to capture specific details about the job steps.

How much reliability do you expect of the data processing platform?

Choose a platform that minimizes single points of failure and supports retries for failed tasks. For example, hosting custom processing logic invoked from Data Factory on Azure Kubernetes Service (AKS) typically offers stronger reliability than hosting it in Azure Logic Apps.

If your data updates infrequently and you handle processing through weekly batch processing, occasional failures may be acceptable. But for real-time AI scenarios, you'll need higher reliability.

Are there any cost constraints?

The goal is to avoid over-engineering and pick a platform that fits your needs today while leaving room to scale. For example, if you don't need Databricks' advanced features, Data Factory might offer a more affordable option. Open-source tools like Airflow or NiFi can further reduce costs.

What are the security requirements on the workflows and on the data that you process?

Security, privacy, and data residency requirements should guide your choice. Ideally, the platform should provide built-in support for such isolation that enables efficient and secure data management. At the very least, make sure the platform:

- Meets regional data residency laws. You might need to run separate pipelines for different regions, such as one for Europe and another for America, to meet local compliance regulations.
- Supports identity and access management (IAM) to ensure that only authorized identities have access to specific jobs or steps within workflows.
- Allows fine-grained access control at the workflow or step level.

Technology options

[Expand table](#)

Function	Recommended Technologies	Alternatives / Complementary Tools
Data cleansing	Azure Data Factory, Azure Databricks, Microsoft Fabric Dataflows	Apache NiFi, Apache Airflow
Data transformation	Azure Databricks, Azure Synapse Analytics, Microsoft Fabric Data Engineering	Azure Data Factory Pipelines
Data enrichment	Azure Document Intelligence in Foundry Tools, Azure OpenAI Service, Azure AI Search	Custom Python APIs or third-party AI services
Workflow orchestration	Azure Data Factory Pipelines, Databricks Jobs	Apache Airflow, Apache NiFi
RAG Workflows	Azure OpenAI Service, Azure AI Search, Azure Databricks	Microsoft Fabric Data Science

Considerations for a search index

A search index stores the contextual or grounding data that's sent to a model's inference endpoint along with the prompt. Index queries are a critical component in preparing the data sent to the model in the inference requests and must deliver low-latency performance.

Unlike batch-oriented ETL pipelines, this index must support real-time inferencing, meaning high performance and reliability are non-negotiable. It's purpose-built for AI workloads and supports capabilities like keyword indexing, filtering, and vector-based search, which go beyond what traditional data stores provide.

The ideal design is a high-performance data store, optimized for reads, that can handle imprecise or fuzzy queries while still returning relevant results. Choose the index technology keeping those points in mind.

What types of search does the search index support?

Every request to the system may result into one or more queries to the index. For retrieval-augmented generation (RAG) and other AI-driven workloads, vector search is a must-have. Vector search allows the system to find semantically similar data points by using embeddings rather than exact keyword matches.

However, combining vector search with full-text search, filtering, and special data types (such as geo-location) makes the index far more powerful.

Your data design should clearly specify which search types are required and how they should work together. For more information, see [Efficient querying in the data design](#).

How does the index handle multimodal data?

AI workloads often deal with data that includes not just text, but also images, audio, or video. The index itself can't directly understand images. So before adding images to the index, they need to either be converted into a text-based representation (using OCR or image captioning), from which embeddings are generated, or vector embeddings can be generated directly from the image using vision models. The index can then perform vector search, allowing semantic queries.

In this use case, the search index should have:

- Vector search support to store and query embeddings (numeric vectors) derived from the image.
- Integration with external APIs and AI services for extracting or enriching data during the indexing process.
- Ability to store extracted fields (text, tags, captions, embeddings) in appropriate schema fields as metadata for search and filtering.

Does the index support automatic update capabilities when the data in the data sources changes?

Automation is key for maintaining data freshness. Select an index that supports automatic updates or incremental refreshes when the underlying data changes.

If the platform doesn't offer this natively, you'll need to implement a custom process to detect and push updates. Offloading this responsibility to the platform can reduce operational overhead and simplify maintenance, especially as data volumes grow.

Can the index perform with large volumes of data?

The index must scale efficiently as data volume grows. For workloads that implement RAG, each document is often split into multiple chunks, which significantly increases the amount of data stored.

Your chosen platform should be able to:

- Scale horizontally as data grows
- Maintain query performance under heavy load

- Store both raw data and related metadata, enrichments, and entities

Does the index have built-in reliability features?

Reliability for the search index should mirror that of your inference endpoint as both are part of the same real-time processing path.

Each step must meet similar uptime and performance expectations. To achieve this, when you choose the data platform, look for:

- High availability and zone redundancy capabilities to survive zone and regional outages.
- Automatic recovery and easy index rebuilding to prevent the use of a corrupted index for inferencing.
- Index aliasing or swap capabilities to enable updates with zero downtime.

Also, understand the system's failure modes or stress indicators, such as throttling. For example, during background reindexing, throughput may drop. The system might normally handle 50 concurrent users but only 30 during that job. Plan job timing and capacity accordingly, accounting for both front-end queries and back-end maintenance tasks.

What are the main cost drivers of this technology?

Search index costs are typically usage-based, so it's important to model your expected data volume, query rate, and throughput.

Most index platforms, such as Azure AI Search, are Platform as a Service (PaaS) offerings, where pricing is abstracted and presented in units of capacity, storage, and feature usage.

Be mindful of:

- Tier pricing and scaling limits
- Extra costs from advanced features (for example, image extraction or skillset enrichment)
- Unused capacity in overprovisioned tiers
- Index complexity (number of indexes and concurrent query limits)

To understand the costs associated with AI Search, see [Plan and manage costs of an AI Search service](#).

Do the security features of the index fulfill your security data design?

Your data design should clearly specify the security and privacy requirements, and your index must fully support them. When working in dev or test environments that use real data, ensure

the index complies with access control and traceability policies. Look for features such as:

- Data masking and PII removal
- Client identity management through Microsoft Entra ID
- Document-level access controls for filtering results based on user identity

If the platform doesn't natively support these, consider implementing query-level filters as a fallback. For more information, see [Security filters for trimming results in AI Search](#).

From a network security perspective, the index should:

- Support egress control and network segmentation
- Integrate with private networks when compute runs in a virtual network
- Use managed identities for authentication through Microsoft Entra ID
- Avoid exposing components directly to the public internet

Embeddings can still expose sensitive information if not properly secured. Risks include embedding inversion (reconstructing original text from vectors), data poisoning (inserting malicious vectors), and unauthorized access to embedding stores or backups. To mitigate these risks, apply security measures like:

- Encryption at rest and in transit
- Strict access controls
- Private network connectivity discussed above
- Monitor embedding endpoints for anomalies or tampering

Similar to other types of data, have processes to remove sensitive or personal data. Treat vector indexes as sensitive data stores that require the same level of security and governance as other production systems.

Technology options

 Expand table

Function	Recommended Technologies	Alternatives / Complementary Tools
Vector search and semantic search	Azure AI Search, Azure Cosmos DB (vector search), Azure Database for PostgreSQL (pgvector)	Pinecone, Weaviate, Chroma, Qdrant
Full-text search and keyword indexing	Azure AI Search	Elasticsearch, Apache Solr, Azure SQL Database Full-Text Search

Function	Recommended Technologies	Alternatives / Complementary Tools
Multimodal data processing	Azure AI Search (with skillsets), Document Intelligence, Azure Vision in Foundry Tools	Custom processing with OpenAI APIs, Amazon Textract
Automatic data refresh and indexing	Azure AI Search (with indexers), Azure Data Factory triggers	Custom polling solutions, Apache NiFi, change data capture
High-availability and reliability	Azure AI Search (zone redundancy), Azure Cosmos DB (global distribution)	Multi-region deployments, load balancers, Azure Traffic Manager
Index aliasing and zero-downtime updates	Azure AI Search (index aliases), Azure Cosmos DB	Blue-green deployment patterns, custom routing logic
Document-level security and access control	Azure AI Search (security filters), Microsoft Entra ID integration	Custom authorization layers, row-level security in databases
Network security and private access	Azure Private Link, Virtual Network integration, Managed Identities	VPN gateways, Azure Firewall, custom network security groups

Training and fine-tuning considerations

When designing your data platform for traditional machine learning (ML) or non-GenAI workloads, your focus shifts from real-time inference to data quality, reproducibility, and environment separation. These workloads rely on well-structured aggregated data and often involve additional layers, such as feature stores and batch inference data stores, to optimize model performance and cost efficiency.

We highly recommend that you understand the principles of good data pipeline design before you explore the technological capabilities that this article describes. For more information, see [Training data design](#).

Do you plan to conduct training with production data?

How you deploy your models determines how tightly production data is coupled with your development environment. There are two main deployment approaches:

- **Model deployment.** The model is trained or tuned using production data during development. This approach can improve model relevance but demands strong security controls, since sensitive data is being used outside production.

- **Code deployment.** The model is trained using non-production data in development and only interacts with real data once it's deployed to production. This method simplifies development security but can increase compute and storage costs, since training may need to be repeated in multiple environments.

Regardless of the approach, your data platform should clearly separate development and production environments, ensuring proper isolation and access control.

Are you prioritizing convenience over functionality?

When choosing a data platform for ML, don't make the decision based solely on notebook support.

Notebooks are great for exploratory data analysis, but they're not a deciding factor for selecting a production-grade data platform. Notebook compute resources usually sit outside the aggregation data store and are integrated with external tools such as Azure Machine Learning or Databricks Workspaces.

Prioritize core capabilities, such as data versioning, governance, scalability, and security,— over convenience features.

How will you process and prepare your data?

In ML workloads, the data processing pattern you choose has a big impact on flexibility and performance.

- ETL (Extract, Transform, Load) – Common in traditional data warehousing, where schema constraints require you to transform data before loading it into the target system.
- ELT (Extract, Load, Transform) – Typical for data lakes or lakehouse architecture, where raw data is loaded first, then transformed later using tools like Python or PySpark.
- EL (Extract, Load) – Common in GenAI and RAG patterns, where you store documents or media first and perform downstream transformations (like text chunking or image extraction) later.

ELT is often preferred since it preserves raw data and allows more flexible transformations during model preparation.

Do you need a feature store?

It's often beneficial to introduce a feature store as an intermediate data layer between your aggregated data store and training environment.

A feature store acts as a catalog of curated features, complete with metadata such as feature lineage, generation time, and source. It's the perfect place to maintain "golden" training data that can be reused across multiple models or experiments.

Managed feature stores, such as the one in Azure Machine Learning, integrate directly with MLflow and other ML lifecycle tools. They enable reproducibility, governance, and version control for your features.

Treat the feature store as a sensitive data store in its own right with proper access controls, encryption, and auditing.

Should you use a batch inference data store?

In some cases, you can improve performance and reduce costs by performing inferencing in batch, that is, pre-computing inference results and storing them for later use instead of calling the model in real time.

This approach can be highly effective when the same queries or predictions are requested repeatedly (for example, generating FAQs or standard recommendations).

Key benefits include:

- Reduced latency and improved user experience, results are served instantly.
- Easier scalability because inference can be batched and distributed offline.
- Enhanced reliability that avoids putting real-time load on the inference endpoint.
- Lower compute costs resulting from batch processing can use lower-tier hardware.
- Built-in pre-validation where results can be verified for accuracy before being exposed to users.

However, this approach works best when a significant percentage of predictions are reused. If your workload involves mostly unique queries, maintaining a batch inference store may not be worth the complexity.

Your batch inference data store should be optimized for read operations, scalable enough to handle large datasets, and integrated with your aggregate data store.

Technologies that fit this pattern include Azure Cosmos DB for fast, globally distributed access, or Azure Table Storage for simpler, lower-cost read-heavy workloads.

Technology options

Function	Recommended technologies	Alternatives / complementary tools
Aggregated data storage	Azure Data Lake Storage Gen2, Microsoft Fabric Lakehouse, Azure Synapse Analytics	Azure Blob Storage, SQL Database, on-premises data warehouse
Data processing and transformation (ETL/ELT)	Azure Data Factory, Azure Databricks (PySpark, SQL), Microsoft Fabric Data Engineering	Apache Airflow, Apache NiFi, Synapse Pipelines
Development and training environment	Azure Machine Learning (with MLflow integration), Azure Databricks Workspaces	JupyterHub, Kubeflow, Amazon SageMaker
Feature store	Azure Machine Learning Feature Store, Databricks Feature Store	Feast (open source), Tecton
Batch inference	Azure Cosmos DB, Azure Table Storage	Azure SQL Database, PostgreSQL, Redis Cache
Model registry and experiment tracking	MLflow (integrated in Azure Machine Learning or Databricks)	Weights & Biases, Neptune.ai, DVC
Orchestration and automation	Azure Data Factory Pipelines, Azure Machine Learning Pipelines	Apache Airflow, Prefect
Security and access control	Microsoft Entra ID (Azure AD), Azure Key Vault, Managed Identities	HashiCorp Vault, AWS IAM

Next steps

Design area: MLOps and GenAIOps

Last updated on 11/21/2025